

Course: Full Stack Web Development

Course Code: 23CM4121

Assignment No.: 1

Student Name: B.Venkata sai laxmi

Roll No.: A24126552074

Date: 30-08-2026

1. Title : **Implement Inheritance Types using Java Script**

2. Aim: Implementing types of inheritances using Java Script.

3. OBJECTIVE :

The objectives of this experiment are:

- To implement five types of inheritance in JavaScript: Single, Multilevel, Hierarchical, Multiple (using Mixins), and Prototype-based inheritance
- To use ES6 class syntax with extends and super keywords for creating parent-child relationships
- To demonstrate method overriding, polymorphism, and code reusability through inheritance hierarchies
- To model real-world scenarios (Employee-Developer-Manager, Animal-Dog-Cat, Vehicle-Car-SUV) using different inheritance patterns
- To simulate multiple inheritance using Object.assign() with mixins since JavaScript doesn't support it directly
- To understand JavaScript's prototype chain and implement inheritance using Object.create()
- To differentiate between inheritance types and choose appropriate patterns for specific use cases
- To create clean, maintainable, and well-documented code following best practices

4. THEORY :

Inheritance:

Inheritance is a fundamental concept in Object-Oriented Programming (OOP) where a child class (subclass) acquires properties and methods from a parent class (superclass).

Types of Inheritance in JavaScript

1. Single Inheritance

One child class inherits from **one** parent class. This is the most basic and common form of inheritance.

Parent Class



Child Class

2. Multilevel Inheritance

Inheritance chain with multiple levels where a child becomes parent for another class. Properties and methods propagate down the chain.

Grandparent Class



Parent Class



Child Class

3. Hierarchical Inheritance

One parent class has **multiple** child classes. All children inherit common functionality from parent while having specialized behavior.

Parent Class



Child1 Child2 Child3

4. Multiple Inheritance (Using Mixins) :

One child inherits from **multiple** parent classes. **NOT SUPPORTED** directly in JavaScript; simulated using **Mixins**.

Parent1 Parent2



Child Class

5. Hybrid inheritance :

Hybrid Inheritance is a combination of two or more inheritance types (single, multilevel, hierarchical, multiple) in one program. It's like mixing different inheritance patterns.

Implementation :

1.simple Inheritance:

```
class Animal {  
  constructor(name) {  
    this.name = name;  
  }  
  eat() {  
    console.log(`${this.name} is eating`);  
  }  
}  
  
class Dog extends Animal {  
  constructor(name, breed) {  
    super(name); // Now this works!  
    this.breed = breed;  
  }  
}
```

```

    }
    bark() {
        console.log(`${this.name} is barking`);
    }
}

const myDog = new Dog("Buddy", "Golden Retriever");
myDog.eat();
myDog.bark();

```

Output :

```

PS C:\Users\VENKATA SAI LAXMI\OneDrive\Desktop\fullstack css advanced> node inheritances.js
Buddy is eating
Buddy is barking

```

2. Multilevel Inheritance :

```

class LivingBeing {
    constructor(name) { // Add constructor
        this.name = name;
    }
    breathe() {
        console.log(`${this.name} is breathing`);
    }
}

#accessing the properties from LivingBeing class

class Animal extends LivingBeing {
    constructor(name) { // Add constructor
        super(name);
    }
    move() {
        console.log(`${this.name} is moving`);
    }
}

class Dog extends Animal {
    constructor(name, breed) { // Add constructor
        super(name);
        this.breed = breed;
    }
}

```

```
bark() {  
    console.log(`${this.name} is barking`);  
}  
}
```

#Create instance and call methods

```
const myDog = new Dog("Buddy", "Golden Retriever");  
myDog.breathe();  
myDog.move();  
myDog.bark();
```

Output :

```
PS C:\Users\VENKATA SAI LAXMI\OneDrive\Desktop\fullstack css advanced> node inheritances.js  
Buddy is breathing  
Buddy is moving  
Buddy is barking
```

3.Hirarcheal Inheritance :

```
class Bird {  
    constructor(name) {  
        this.name = name;  
    }  
    fly() {  
        console.log(`${this.name} is flying`);  
    }  
}  
  
class Sparrow extends Bird {  
    constructor(name) {  
        super(name);  
    }  
    chirp() {  
        console.log(`${this.name} is chirping`);  
    }  
}  
  
class Eagle extends Bird {  
    constructor(name) {  
        super(name);  
    }  
    hunt() {
```

```

    console.log(`${this.name} is hunting`);
  }
  // Method Overriding
  fly() {
    console.log(`${this.name} is soaring high`);
  }
}

// Create instances and call methods
const sparrow = new Sparrow("Jack");
const eagle = new Eagle("Sky");

// Sparrow methods
sparrow.fly(); // Jack is flying (from Bird)
sparrow.chirp(); // Jack is chirping (from Sparrow)
console.log(); // Empty line

// Eagle methods
eagle.fly(); // Sky is soaring high (overridden)
eagle.hunt(); // Sky is hunting (from Eagle)

```

Output :

```

PS C:\Users\VENKATA SAI LAXMI\OneDrive\Desktop\fullstack css advanced> node inheritances.js
Jack is flying
Jack is chirping

Sky is soaring high
Sky is hunting

```

4. Multiple Inheritance :

```

// Mixin 1
const CanSwim = {
  swim() {
    console.log(`${this.name} is swimming`);
  },
  dive() {
    console.log(`${this.name} is diving`);
  }
};

// Mixin 2
const CanFly = {

```

```
fly() {
  console.log(`${this.name} is flying`);
},
land() {
  console.log(`${this.name} is landing`);
}
};

// Mixin 3
const CanWalk = {
  walk() {
    console.log(`${this.name} is walking`);
  },
  run() {
    console.log(`${this.name} is running`);
  }
};

// Class
class Duck {
  constructor(name) {
    this.name = name;
  }
  quack() {
    console.log(`${this.name} says Quack!`);
  }
}

// Apply multiple mixins
Object.assign(Duck.prototype, CanSwim, CanFly, CanWalk);

const duck = new Duck("Donald");
duck.swim(); // Donald is swimming
duck.fly(); // Donald is flying
duck.walk(); // Donald is walking
duck.quack(); // Donald says Quack!
```

Output :

```
PS C:\Users\VENKATA SAI LAXMI\OneDrive\Desktop\fullstack_css_advanced> node inheritances.js
Donald is swimming
Donald is flying
Donald is walking
Donald says Quack!
```

5.Hybrid Inheritance :

#Using multilevel + multiple

```
class Animal {
  constructor(name) {
    this.name = name;
  }
  eat() {
    console.log(`${this.name} is eating`);
  }
}
class Bird extends Animal {
  chirp() {
    console.log(`${this.name} is chirping`);
  }
}
class FlyingBird extends Bird {
  fly() {
    console.log(`${this.name} is flying`);
  }
}
```

#MIXINS (For Multiple Inheritance)

```
const CanSwim = {
  swim() {
    console.log(`${this.name} is swimming`);
  }
};
const CanHunt = {
  hunt() {
    console.log(`${this.name} is hunting`);
  }
}
```

```
};  
  
class Duck extends FlyingBird {  
  constructor(name) {  
    super(name);  
  }  
  quack() {  
    console.log(`${this.name} says Quack!`);  
  }  
}  
  
// Applying mixins  
Object.assign(Duck.prototype, CanSwim, CanHunt);  
  
const duck = new Duck("Donald");  
  
duck.eat();  
duck.chirp();  
duck.fly();  
duck.swim();  
duck.hunt();  
duck.quack();
```

Output :

```
PS C:\Users\VENKATA SAI LAXMI\OneDrive\Desktop\fullstack css advanced> node inheritances.js  
Donald is chirping  
Donald is flying  
Donald is swimming  
Donald is hunting  
Donald says Quack!
```